

Neural networks

Some history

- 80's : neural networks appeared! lots of excitement
- time: methodology stabilized
- time: SVMs, random forests and other methods appeared they outperformed neural networks.

2010s : neural networks resurfaced with new architectures, massive datasets made it possible to train them

environmental Applications

A small list of prominent use cases of deep learning and neural networks to environmental problems, ecology,

In this lesson we will present what are the components and structure of a single layer neural network, a multilayer neural network, and a convolutional neural network.

Single layer neural network.

Context: p features $X = (X_1, \dots, X_p)$ for which we want to predict a response variable Y .

The neural network is doing the same process we have examined previously, come up with a function $f(X)$ to predict Y .

Let's see a small example where Y is a quantity we are trying to predict.

The structure of the NN is

Figure 10.1 from ISLP goes here, or a recreation of it.

The process is as follows:

- The p features $X_1, \dots, X_p$ make up the input layer this corresponds to putting an observation $(X_1, \dots, X_p)$ through the network.
- Each neuron in the hidden layer takes in all the features and computes an activation.

In this example we have K neurons, so we will compute K activations

$$A_1, \dots, A_K$$

- An activation A_i in the i -th neuron is computed in three steps:

FIRST: each activation has a bias w_{i0} and weights $w_{i1}, \dots, w_{ip}$ (one weight per feature). These are used to compute the following quantity with the features $w_{i0} + w_{i1}X_1 + \dots + w_{ip}X_p$.

SECOND: This quantity goes through an activation function $g(z)$ which makes every value be between 0 and 1.

There's two popular activation functions:

- sigmoid activation function: $g(z) = \frac{e^z}{1 + e^z}$
- ReLU activation function: $g(z) = \begin{cases} 0, & \text{if } z < 0 \\ z, & \text{if } z \geq 0 \end{cases}$

Add plots for activation functions.

The activation at the neuron is the value

$$A_K = g(w_{K0} + w_{K1}X_1 + \dots + w_{Kp}X_p)$$

and it depends on $p+1$ parameters: p weights + 1 bias per neuron.

- Once all the activations $A_1, \dots, A_K$ at the hidden layer are computed, the final model is given by

$$f(X_1, \dots, X_p) = \beta_0 + \beta_1 A_1 + \dots + \beta_K A_K,$$

where $\beta_0, \beta_1, \dots, \beta_K$ are an additional set of $K+1$ parameters.

Fitting the neural network is the process of estimating the weights and biases for each neuron and the weights β_i using the training data so error is minimized.

Example: This would be an example calculating the number of parameters in a NN with 4 features and 5 neurons in the hidden layer.

Multilayer Neural Networks

Modern neural networks typically have more than one hidden layer and often many neurons per layer (hundreds).

The input layer is still the same: the features $X_1, \dots, X_p$.

These go on to the first hidden layer where each neuron produces an activation $A_1^{(1)}, \dots, A_{K_1}^{(1)}$:

$$A_i^{(1)} = g(w_{i0}^{(1)} + \sum_{k=1}^p w_{ik}^{(1)} X_k).$$

The second hidden layer treats the activations $A_1^{(1)}, \dots, A_{K_1}^{(1)}$ as an input layer and computes new activations

$$A_i^{(2)} = g(w_{i0}^{(2)} + \sum_{k=1}^{K_1} w_{ik}^{(2)} A_k^{(1)}).$$

This process repeats itself as you move forward with the layers, with each layer adding more weights to the model.

To have a better sense of how a multi-layer network works and how these can be used for classification, we will watch the 3 blue, 1 brown video.

As you watch it, try to answer the following questions.

Convolutional Neural Networks.

In these networks layers do additional work to identify low-level features within the image, such as small edges, patches of color, textures, etc.

The low-level features are then combined to form higher level features, like ears, eyes, etc. Eventually, the presence or absence of these higher level features contribute to the probability of belonging to a class or not.

Convolution and maxpool layer

A convolution filter is a template that finds or highlights local features in an image.

Example:

$$\begin{array}{|c|c|c|c|c|} \hline 1 & 1 & 1 & 1 & 1 \\ \hline 0 & 1 & 0 & 1 & 0 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline 0 & 1 & 0 & 1 & 0 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \hline \end{array} \times \begin{array}{|c|c|} \hline 1 & 1 \\ \hline 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 2 & 2 & 2 & 2 \\ \hline 1 & 1 & 1 & 1 \\ \hline 2 & 2 & 2 & 2 \\ \hline 1 & 1 & 1 & 1 \\ \hline \end{array}$$
$$\times \begin{array}{|c|c|} \hline 1 & 0 \\ \hline 1 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 1 & 2 & 1 & 2 \\ \hline 1 & 2 & 1 & 2 \\ \hline 1 & 2 & 1 & 2 \\ \hline 1 & 2 & 1 & 2 \\ \hline \end{array}$$

The convolved image highlights regions in the original image that resemble the convolution filter.

A convolution layer aggregates a large number of these convolution filters, each of which determines whether a particular feature is present. In a CNN the filters are learned for the specific classification task!. The filter weights are a new set of parameters going from an input layer to a hidden layer.

A pooling layer provides a way to condense a large image into a smaller summary image. A popular method to do this is using maxpooling, where each non-overlapping 2×2 block of pixels is condensed into the maximum value within that 2×2 block.

Assembling a CNN

Here I will just insert a diagram like ISLP 10.8.